Desarrollo de Aplicaciones Web

JavaScript. Introducción.

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces

JavaScript

- Es un lenguaje de programación interpretado, originalmente creado en 1995 por *Brendan Eich* de la empresa *Netscape* para añadir interactividad a las páginas web abiertas con su navegador *Netscape Navigator 2.0*.

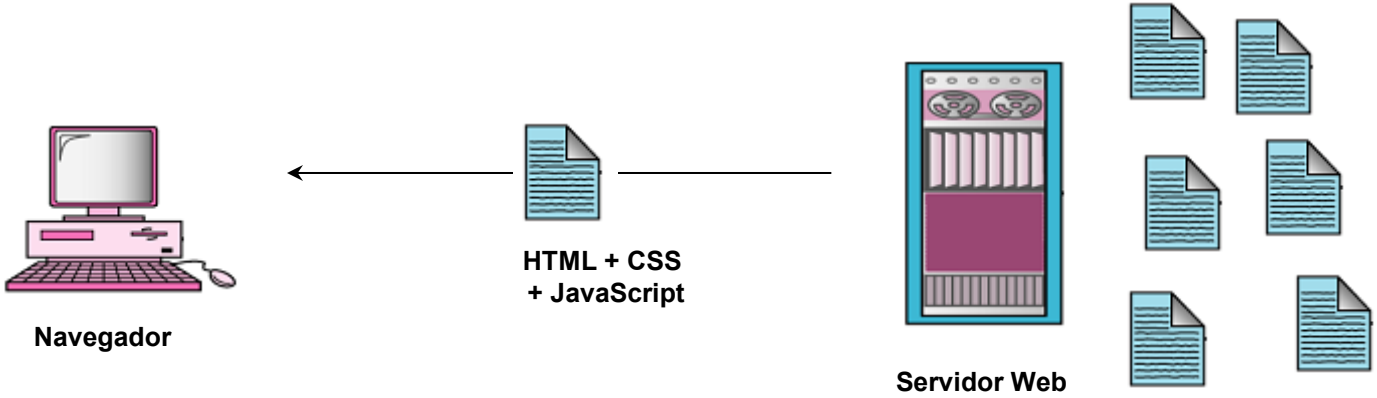
- El nombre original fue *Mocha*, siendo rebautizado posteriormente como *JavaScript*, mediante acuerdo con *Sun Microsystems*. No es *Java*.

- En 1997 comenzó el proceso de estandarización, primero en ECMA (*European Computer Manufacturers Association)* con el nombre *ECMAScript* y posteriormente también estándar ISO (*International Organization for Standardization*).

- Con su aparición se introduce el concepto de HTML dinámico, DHTML.

JavaScript

Javascript aparece embebido en el código HTML, siendo interpretado (no compilado) por el navegador (lado cliente) al descargar la página.



Nota.- *en la actualizad existe la posibilidad de ejecutar código javascript en el lado servidor, mediante el entorno de ejecución "Node.js", pero no será tratado en este tema.*

JavaScript

Motores JavaScript:

Navegador	Script Engine
Apple Safari	Nitro
Google Chrome	V8
Microsoft IE	Chakra
Mozilla Firefox	JägerMonkey
Opera	Carakan

Integración con HTML

Debe ir dentro de etiquetas <script>… </script>

Se ejecuta, en el punto en el que aparece escrito dentro del código HTML

Existe la posibilidad de incluirlo como página interna o externa (fichero.js) y ser invocado a lo largo de la página HTML.

En el caso de páginas externas se incluye en la página con:

<script src="codigo.js"></script>

Con el fin de mejorar la carga de la página, se aconseja ponerlo al final, antes de </body>

Otros lenguajes de Script

Microsoft desarrolló dos versiones propias para el IE:

- *JScript* → versión *ECMAScript* similar al *JavaScript* de *Netscape*, pero con algunas diferencias en el modelo de objetos.

- *VBScript* → incompatible con navegadores diferentes a IE y con sintaxis basada en el *Visual Basic*.

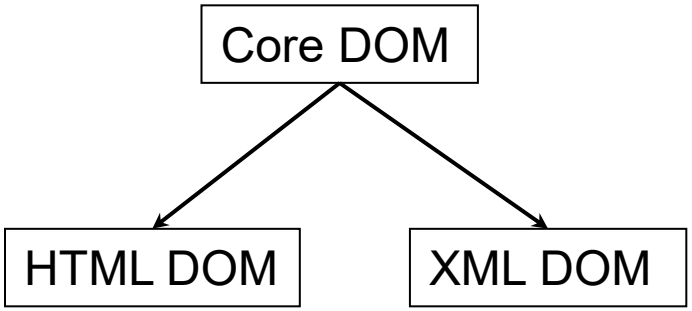
La diferencia en el modelo de objetos entre *JScript* y *JavaScript*, produjo incompatibilidades → W3C definió el *HTML DOM*, con el fin de aunar criterios.

DOM

W3C, a través del "*Document Object Model*" (DOM), define un mecanismo estándar de acceso a documentos HTML, XML y SVG permitiendo de esta manera que programas y scripts puedan modificar su contenido, estructura y/o estilo.

W3C define tres partes separadas:

- **Core DOM →** modelo estándar para cualquier documento estructurado
- **XML DOM →** modelo estándar para cualquier documento XML
- **HTML DOM →** modelo estándar para cualquier documento HTML



DOM

W3C, a través del "*Document Object Model*" (DOM), define un mecanismo estándar de acceso a documentos HTML y XML, permitiendo de esta manera que programas y scripts puedan modificar su contenido, estructura y/o estilo.

W3C define tres partes separadas:

- **Core DOM** → modelo estándar para cualquier documento estructurado

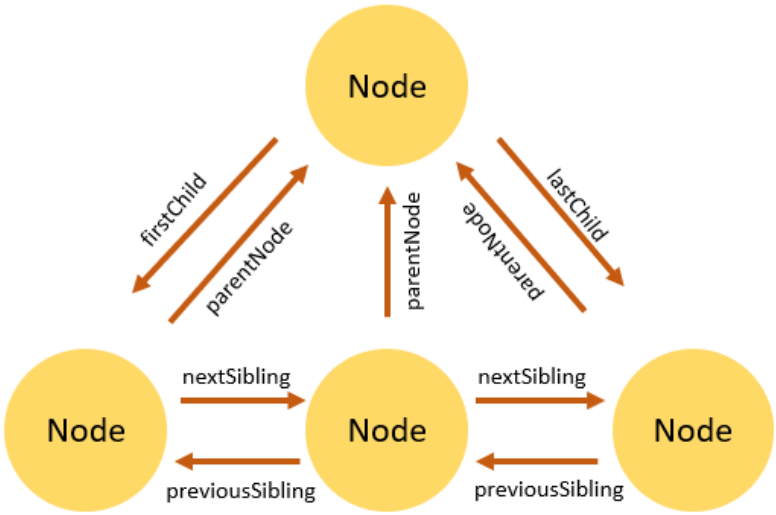
Ref	Tipo de Nodo
1	**ELEMENT_NODE**
2	**ATTRIBUTE_NODE**
3	**TEXT_NODE**
4	CDATA_SECTION_NODE
5	ENTITY_REFERENCE_NODE
6	ENTITY_NODE
7	PROCESSING_INSTRUCTION_NODE
8	**COMMENT_NODE**
9	DOCUMENT_NODE
10	DOCUMENT_TYPE_NODE
11	DOCUMENT_FRAGMENT_NODE
12	NOTATION_NODE

DOM

W3C, a través del "*Document Object Model*" (DOM), define un mecanismo estándar de acceso a documentos HTML y XML, permitiendo de esta manera que programas y scripts puedan modificar su contenido, estructura y/o estilo.

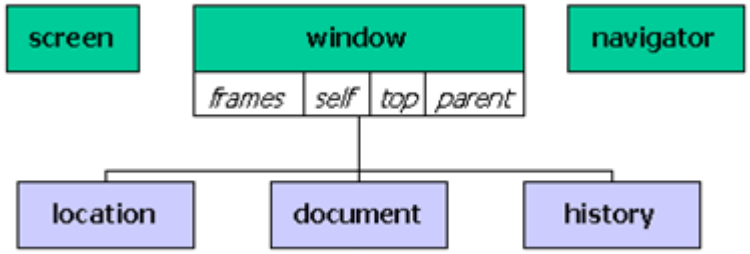
W3C define tres partes separadas:

- **Core DOM →** modelo estándar para cualquier documento estructurado



HTML DOM

Mediante JavaScript, es posible tener acceso a los diferentes objetos relacionados con un documento HTML, permitiendo la realización de todo tipo de modificaciones.



Nota → *document* es una propiedad del objeto *window*, que representa al navegador.

HTML DOM

Respecto a los objetos del DOM, utilizados para acceder a las diferentes partes del documento serían los siguientes:

Objeto DOM	Descripción	Hereda de
Document	Representa el documento completo	*Node*
Element	Representa cualquier etiqueta del documento	*Node*
Attr	Representa cualquier atributo de las etiquetas	*Node*
Text	Representa los Nodos Texto del documento	*Node*
Comment	Representa los comentarios del documento	*Node*
HTMLDocument	Representa el documento HTML completo	*Document*
HTMLElement	Representa cualquier etiqueta HTML del documento	*Element*
Style	Representa los estilos asociados a las etiquetas del documento	*CSSStyleDeclaration*
Event	Representa los eventos asociados a las etiquetas del documento	*Object*

Nota → En un documento HTML *document* es una instancia de la clase HTMLDocument.

HTML DOM

A mayores existen otros objetos del DOM, necesarios para la gestión de los accesos:

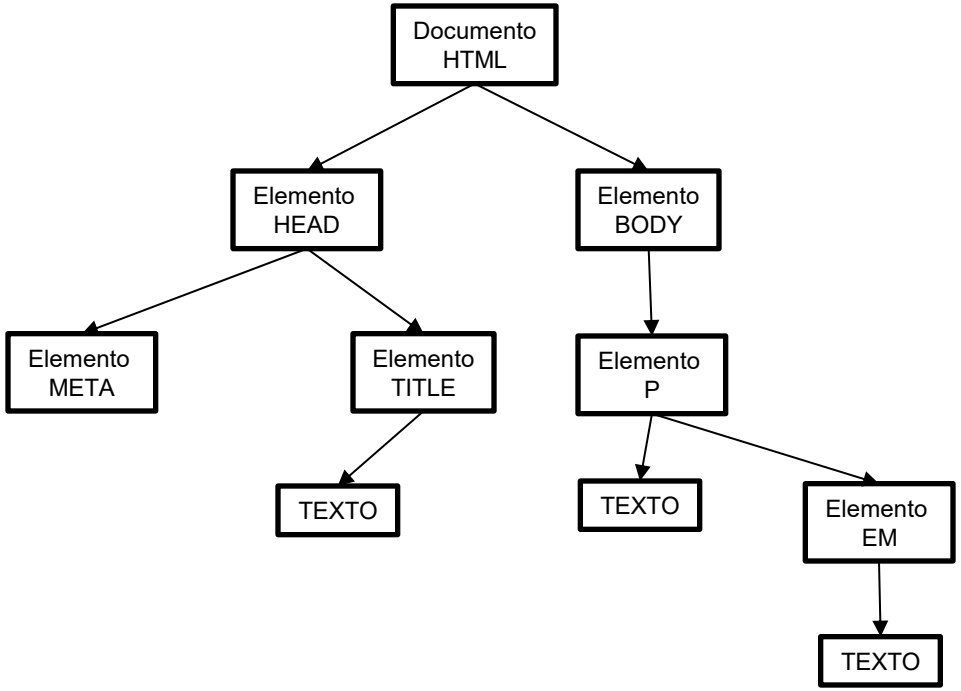
Objeto HTML DOM	Descripción	Retornado por	Propiedades/ Métodos/Eventos
HTMLCollection	Lista de etiquetas HTML.	*Métodos javascript de acceso a etiquetas*	Prop/Met
NodeList	Lista de objetos nodo del documento.	*Métodos javascript de acceso a nodos y etiquetas*	Prop/Met
NamedNodeMap	Colección de atributos de etiquetas	*.attributes*	Prop/Met
DOMTokenList	Lista de clases de una etiqueta	*.classList*	Prop/Met

HTML DOM

document → Representa al propio documento que se está mostrando en la ventana del navegador.

Basados en HTML DOM, los navegadores transforman automáticamente las etiquetas de un documento HTML en una estructura arborescente, cuyos nodos son accesibles.

```
<html>
  <head>
     <meta  ----------------- />
     <title> -------------------- </title>
  </head>
  <body>
      <p> ------ <em> ----- </em></p>
  </body>
</html>
```



HTML DOM

document → Representa al propio documento que se está mostrando en la ventana del navegador.

La transformación automática de la página en un árbol sigue las reglas:

- Las etiquetas HTML se transforman en dos nodos:
  - La propia etiqueta.
  - Texto, que aparece como hijo de nodo que representa la etiqueta en cuestión.
- Si una etiqueta HTML se encuentra dentro de otra, se sigue el procedimiento anterior, siendo los nodos generados hijos del que representa a la etiqueta en cuestión.

HTML DOM

document → Representa al propio documento que se está mostrando en la ventana del navegador.

Etiqueta *vs* Elemento

Concepto	Qué es	Ejemplo
Etiqueta	Texto literal en el HTML que lee el navegador	*<p>*
Elemento	Objeto del DOM creado a partir de esa etiqueta	*HTMLParagraphElement*

HTML DOM

Mediante DOM, JavaScript obtiene capacidad completa para crear todo tipo de efectos DHTML y así poder:

- Cambiar todos las etiquetas HTML de la página.
- Cambiar todos los atributos HTML de la página.
- Cambiar todos los estilos CSS de la página.
- Eliminar etiquetas y atributos HTML.
- Añadir nuevas etiquetas y atributos HTML.
- Reaccionar a todos los eventos HTML de la página.
- Crear nuevos eventos HTML en la página.
- …

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Acceso a elementos individuales:

- *getElementById(nombre)* → devuelve el elemento HTML cuyo atributo id coincide con el parámetro pasado.

- *querySelector(CSS_Selector)* → devuelve el primer elemento de la página coincidente con el selector CSS pasado.

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Acceso a elementos individuales:

```
<!DOCTYPE html>
<html>
    <body>
        <p class="daw">ETSE</p>
        <p class="daw">Desarrollo de Aplicaciones Web</p>

        <script>
            document.querySelector(".daw").style.color= "red";
        </script>
    </body>
</html>
```

DOM Interface: Acceso a elementos

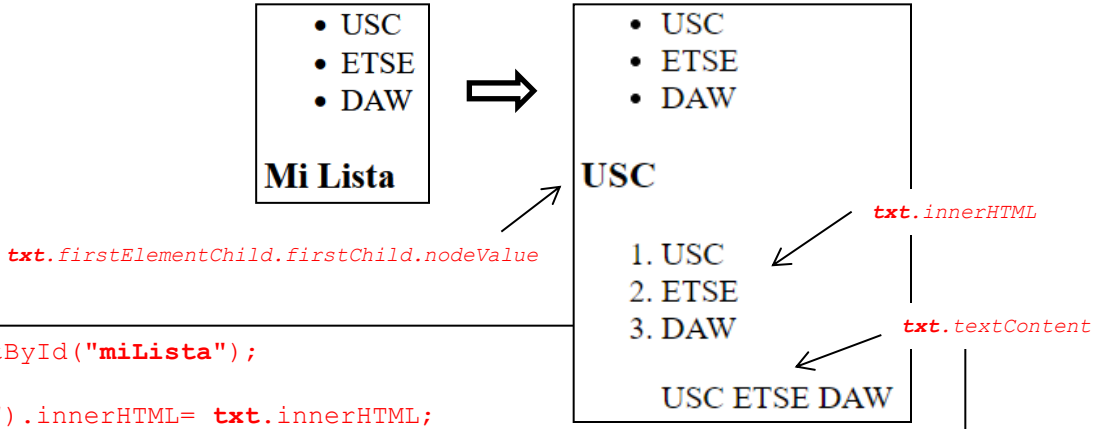
HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para acceder al contenido de un nodo:

- *nodeValue* → devuelve el contenido del nodo actual.
- *innerHTML* → devuelve el contenido de los hijos del nodo actual.
- *textContent* → devuelve el contenido de texto de los hijos del nodo actual.

```
<ul id="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
</ul>

<h3 id="Nodo">Mi Lista</h3>
<ol id="Lista"></ol>
<ol id="Lista1"></ol>
```




```
const txt = document.getElementById("miLista");

document.getElementById("Lista").innerHTML= txt.innerHTML;
document.getElementById("Lista1").innerHTML= txt.textContent;
document.getElementById("Nodo").innerHTML= txt.firstElementChild.firstChild.nodeValue;
```

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para acceder al contenido de un nodo:

- *nodeValue* → devuelve el contenido del nodo actual
- *nodeName* → devuelve el nombre del nodo actual.
- *nodeType* → devuelve el tipo del nodo actual, según tabla

Tipo	Significado
1	Nodo Elemento
2	Nodo Artributo
3	Nodo Texto
8	Nodo Comentario

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para acceder al contenido de un nodo:

- *nodeValue* → devuelve el contenido del nodo actual
- *nodeName* → devuelve el nombre del nodo actual.
- *nodeType* → devuelve el tipo del nodo actual, según tabla

```
<ul id="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
</ul>

<p id="otroNodo"></p>
```

```
const x =document.getElementById("miLista");

let text = "";
text += "Name: " + x.nodeName + " - ";
text += "Value: " + x.nodeValue + " - ";
text += "Type: " + x.nodeType;
document.getElementById("otroNodo").innerHTML = text;
```

- USC
- ETSE
- DAW

Name: UL - Value: null - Type: 1

nodeType

nodeName *nodeValue*

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Acceso a conjuntos de elementos:

- *getElementsByClassName(nombre)* → devuelve una *HTMLCollection* con todos los elementos de la página cuyo nombre de clase sea igual al parámetro pasado.

- *getElementsByName(nombre)* → devuelve una *HTMLCollection* con todos los elementos de la página cuyo atributo *name* sea igual al parámetro pasado.

- *getElementsByTagName(HTML_tag)* → devuelve una *HTMLCollection* con todos los elementos de la página cuya etiqueta sea igual al parámetro pasado.

- *querySelectorAll(CSS_Selector)* → devuelve un *NodelList* con todos los elementos de la página cuya etiqueta coincida con el selector CSS pasado.

DOM Interface: Acceso a elementos

NodeList → Objeto que representa una colección de nodos sobre los que se puede iterar.

HTMLCollection → Objeto que representa una colección de elementos HTML

Similitudes:

- Son listas tipo array de objetos.
- Tienen una propiedad de longitud (length).
- Proporcionan un índice (0, 1, 2, 3, 4, ...) para acceder a cada elemento.

Diferencias:

- Se puede acceder a los elementos de *HTMLCollection* por su nombre, id o número de índice.
- A los elementos de *NodeList* sólo se puede acceder por su número de índice.
- Sólo el objeto *NodeList* puede contener nodos de atributos y nodos de texto.

Nota 1 *→ A mayores, la diferencia entre un HTMLCollection y un NodeList es que uno es "vivo" y el otro es estático. Esto significa que cuando se añade un elemento al DOM, un nodo "vivo" reconocerá el nuevo elemento mientras que un nodo estático no lo hará.*

Nota 2 *→ algunos navegadores cambian el tipo de dato (NodeList/HTMLCollection) que devuelven los métodos DOM.*

DOM Interface: Acceso a elementos

NodeList → Objeto que representa una colección de nodos sobre los que se puede iterar.

```
<ul class="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
  <li>POO</li>
  <li>IA</li>
</ul>

<p class="otraLista0"></p>
<p class="otraLista1"></p>
<p class="otraLista2"></p>
<ul class="otraLista3"></ul>
<ul class="otraLista4"></ul>
```

- USC
- ETSE
- DAW
- POO
- IA

Tipo de Objeto → [object NodeList]

Número de elementos → 3

Primer elemento de la colección → [object HTMLUListElement]

Contenido en texto del primer elemento → USC ETSE DAW POO IA

- USC
- ETSE
- DAW
- POO
- IA

Contenido del primer elemento

```
const miLista= document.querySelectorAll('ul');

document.querySelector(".otraLista0").innerHTML= miLista;
document.querySelector(".otraLista1").innerHTML= miLista.length
document.querySelector(".otraLista2").innerHTML= miLista[0]
document.querySelector(".otraLista3").innerHTML= miLista[0].textContent
document.querySelector(".otraLista4").innerHTML= miLista[0].innerHTML
```

DOM Interface: Acceso a elementos

HTMLCollection → Objeto que representa una colección de elementos HTML

```
<ul class="miLista">
  <li>USC</li>
  <li>ETSE</li>
  <li>DAW</li>
  <li>POO</li>
  <li>IA</li>
</ul>

<p class="otraLista0"></p>
<p class="otraLista1"></p>
<p class="otraLista2"></p>
<ul class="otraLista3"></ul>
<ul class="otraLista4"></ul>
```

- USC
- ETSE
- DAW
- POO
- IA

Tipo de Objeto → [object HTMLCollection]

Número de elementos → 3

Primer elemento de la colección → [object HTMLUListElement]

Contenido en texto del primer elemento → USC ETSE DAW POO IA

- USC
- ETSE
- DAW
- POO
- IA

Contenido del primer elemento

```
const miLista_1= document.getElementsByTagName('ul');

document.querySelector(".otraLista0").innerHTML= miLista_1;
document.querySelector(".otraLista1").innerHTML= miLista_1.length;
document.querySelector(".otraLista2").innerHTML= miLista_1[0];
document.querySelector(".otraLista3").innerHTML= miLista_1.item(0).textContent;
document.querySelector(".otraLista4").innerHTML= miLista_1[0].innerHTML;
```

DOM Interface: Acceso a elementos

HTMLCollection vs NodeList → "Live" vs Estático

```html
<h3>Mi Lista</h3>
<ul id="miLista">
  <li class="item">USC</li>
  <li class="item">ETSE</li>
  <li class="item">DAW</li>
</ul>

<h3>Tamaño Antes</h3>
<p id="nListAntes"></p>
<p id="nListUlAntes"></p>
<p id="nColAntes"></p>
<p id="nColUlAntes"></p>

<h3>Tamaño Después</h3>
<p id="nListDespues"></p>
<p id="nListUlDespues"></p>
<p id="nColDespues"></p>
<p id="nColUlDespues"></p>
```

```javascript
function showItems(items){

    let txt= "";
    for (let i=0; i<items.length-1; i++){
        txt+= items.item(i).textContent + ", ";
    }
    txt+= items.item(items.length-1).textContent;

    return(txt);
}
```

```javascript
const miLista = document.getElementById('miLista');

const itemsByQuerySelector = document.querySelectorAll('.item');
const itemsByClassName = document.getElementsByClassName('item');

document.getElementById("nListAntes").innerHTML= 'NodeList: ' + itemsByQuerySelector.leng
document.getElementById("nListUlAntes").innerHTML= showItems(itemsByQuerySelector);
document.getElementById("nColAntes").innerHTML= 'HTMLCollection: ' + itemsByClassName.len
document.getElementById("nColUlAntes").innerHTML= showItems(itemsByClassName);

miLista.innerHTML += '<li class="item">POO</li>';

document.getElementById("nListDespues").innerHTML= 'NodeList: '+ itemsByQuerySelector.length;
document.getElementById("nListUlDespues").textContent= showItems(itemsByQuerySelector);
document.getElementById("nColDespues").innerHTML= 'HTMLCollection: ' + itemsByClassName.length;
document.getElementById("nColUlDespues").innerHTML= showItems(itemsByClassName);
```

itemsByQuerySelector

itemsByClassName

Mi Lista

- USC
- ETSE
- DAW
- POO

Tamaño Antes

NodeList: 3

USC, ETSE, DAW

HTMLCollection: 3

USC, ETSE, DAW

Tamaño Después

NodeList: 3

USC, ETSE, DAW

HTMLCollection: 4

USC, ETSE, DAW, POO

blo García Tahoces

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Métodos para crear/eliminar el contenido de un nodo:

- *createElement()* → inserta un nuevo nodo en el DOM.
- *createTextNode()* → inserta un nuevo nodo de texto en el DOM.
- *appendChild() / removeChild()* → inserta/elimina un nodo hijo en el DOM

```
<h2 id="titulo"></h2>
```

```
var e= document.createElement("p");
var t= document.createTextNode("Universidad de Santiago de Compostela");
e.appendChild(t);
document.body.appendChild(e);

e1=document.getElementById("titulo")
e1.innerHTML = "Desarrollo de Aplicaciones Web";
```

Desarrollo de Aplicaciones Web

Universidad de Santiago de Compostela

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para referenciar nodos:

- *parentNode* → selecciona el nodo padre del nodo actual.
- *previousSibling / nextSibling* → selecciona el nodo previo/siguiente al nodo actual.
- *firstChild / lastChild*→ selecciona el primer / último nodo hijo al nodo actual.
- *firstElementChild* → selecciona el primer elemento hijo del nodo actual.
- chilNodes → retorna los nodos hijos del nodo actual.
- children → retorna los elementos hijos del nodo actual.

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para referenciar noc

```
<h1>Prueba Nodos</h1>
<p id= "titulo">Tipo Nodo</p><!-- Prueba Nodos -->

<p id="nodo_1"></p>
<p id="nodo_2"></p>
<p id="nodo_3"></p>
<p id="nodo_4"></p>
```

Prueba Nodos

Tipo Nodo

<p> con id= "titulo"

Name: P - Value: null - Type: 1

firstChild

Name: #text - Value: Tipo Nodo - Type: 3

nextSibling

Name: #comment - Value: Prueba Nodos - Type: 8

Attributes[0]

Name: id - Value: titulo - Type: 2

```
const x= document.getElementById("titulo");
const y= x.firstChild;
const z= x.nextSibling;
const a= x.attributes[0];

let txt_1 = "";
txt_1 += "Name: " + x.nodeName + " - ";
txt_1 += "Value: " + x.nodeValue + " - ";
txt_1 += "Type: " + x.nodeType;
document.getElementById("nodo_1").innerHTML = txt_1;

let txt_2 = "";
txt_2 += "Name: " + y.nodeName + " - ";
txt_2 += "Value: " + y.nodeValue + " - ";
txt_2 += "Type: " + y.nodeType;
document.getElementById("nodo_2").innerHTML = txt_2;

let txt_3 = "";
txt_3 += "Name: " + z.nodeName + " - ";
txt_3 += "Value: " + z.nodeValue + " - ";
txt_3 += "Type: " + z.nodeType;
document.getElementById("nodo_3").innerHTML = txt_3;

let txt_4 = "";
txt_4 += "Name: " + a.nodeName + " - ";
txt_4 += "Value: " + a.nodeValue + " - ";
txt_4 += "Type: " + a.nodeType;
document.getElementById("nodo_4").innerHTML = txt_4;
```

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para referenciar nodos:

```
<body>
    <h1>Prueba Nodos</h1>
    <p id= "titulo">Tipo Nodo 1</p>
    <p>Tipo Nodo 2</p><p>Tipo Nodo 3</p>
    <!-- Prueba Nodos -->
    <p id="nodo"></p>
    <script> ... <script>
</body>
```

```
const nodeList = document.body.childNodes;

let text = "";
for (let i = 0; i < nodeList.length; i++) {
  text += nodeList[i].nodeName + "<br>";
}
document.getElementById("nodo").innerHTML = text;
```

Prueba Nodos

Tipo Nodo 1

Tipo Nodo 2

Tipo Nodo 3

#text
H1
#text
P
#text
P
P
#text
#comment
#text
P
#text
SCRIPT

DOM Interface: Acceso a elementos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Propiedades para referenciar nodos:

```
<body>
    <h1>Prueba Nodos</h1>
    <p id= "titulo">Tipo Nodo 1</p>
    <p>Tipo Nodo 2</p><p>Tipo Nodo 3</p>
    <!-- Prueba Nodos -->
    <p id="nodo"></p>
    <script> ... <script>
</body>
```

```
const nodeList = document.body.children;

let text = "";
for (let i = 0; i < nodeList.length; i++) {
  text += nodeList[i].nodeName + "<br>";
}
document.getElementById("nodo").innerHTML = text;
```

Prueba Nodos

Tipo Nodo 1

Tipo Nodo 2

Tipo Nodo 3

H1
P
P
P
P
SCRIPT

DOM Interface: Acceso a atributos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los atributos de las etiquetas del documento.

Propiedades y Métodos para acceder atributos:

- *element.attributes* → devuelve los atributos de un elemento en forma de colección de objetos atributo (Attr) denominada *NamedNodeMap*.
- *element.getAttribute()* → devuelve un string con el valor del atributo pasado como parámetro.
- *element.getAttributeNames()* → devuelve un array con el nombre de los atributos del elemento.
- *element.getAttributeNode()* → devuelve el nodo del atributo pasado como parámetro, en forma de objeto atributo (Attr).

DOM Interface: Acceso a atributos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los atributos de las etiquetas del documento.

Objetos asociados:

Objeto	Propiedad	Devuelve
Attr	name	Nombre del atributo
	value	Valor del atributo
	nodeType	Tipo de nodo (2)
	ownerParent	Nodo padre

Objeto	Propiedad	Método
NamedNodeMap	length	getNamedItem()
		item()
		removeNamedItem()
		setNamedItem()

Attr → Objeto que representa un atributo de un elemento
NamedNodeMaps → Objeto que representa una colección de objetos Attr

DOM Interface: Acceso a atributos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado.

```
<h1>Atributos DOM</h1>
<img id="myImg" alt="logoUSC" src="logo.png" width="37" height="25">
<p>Los atributos de la imagen son:</p>
<p id="uno"></p>
<p id="dos"></p>
<p id="tres"></p>
```

Atributos DOM

USC

Los atributos de la imagen son:

atr → id= myImg alt= logoUSC src= logo.png width= 37 height= 25

.getAttribute("src") → logo.png

.getAttributeNode("alt").value → logoUSC

```
<script>
const nodeMap = document.getElementById("myImg").attributes;
let atr= "";
for (let i = 0; i < nodeMap.length; i++) {
  atr+= nodeMap[i].name + "= " + nodeMap[i].value + " ";
}
document.getElementById("uno").innerHTML= atr;
document.getElementById("dos").innerHTML= myImg.getAttribute("src");
document.getElementById("tres").innerHTML= myImg.getAttributeNode("alt").value;
</script>
```

DOM Interface: Acceso a atributos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los nodos del árbol generado. Algunos de estos son:

Métodos para manipular atributos:

- *element.hasAttribute()* → comprueba si existe el atributo pasado como parámetro.
- *element.Attribute()* → modifica el valor del atributo pasado como parámetro.
- *element.removeAttribute()* → elimina el valor del atributo pasado como parámetro.

```
<script>
        var enlace = document.getElementById("miEnlace");
        enlace.setAttribute("href", "https://www.usc.es");
</script>
```

DOM Interface: Acceso a estilos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los estilos insertados en el documento:

Propiedad para acceder a los estilos:

- *style* → Permite acceder al estilo del elemento seleccionado.

```
<body>
   <p id="miP">Este es un párrafo.</p>
   <p id="miP1">Este es otro párrafo.</p>
</body>
```

Este es un párrafo.

Este es otro párrafo.

```
<script>
  document.getElementById("miP").style.color = "magenta";
  document.getElementById("miP1").style.border = "thick solid blue";
</script>
```

DOM Interface: Acceso a estilos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los estilos insertados en el documento:

Propiedad para acceder a los estilos:

- *style* → Permite acceder al estilo del elemento seleccionado.

Note → *.style sólo retorna los estilos introducidos en línea*

Método para acceder a todos los estilos:

- *window.getComputedStyle(element)* → Permite acceder a todos los estilos relativos al elemento seleccionado.

Tipo de objeto devuelto (en ambos casos):

- *CSSStyleDeclaration*

DOM Interface: Acceso a estilos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los estilos insertados en el documento:

Objeto CSSStyleDeclaration:

Objeto	Propiedad	Método
CSSStyleDeclaration	cssText	getPropertyPriority()
	length	getPropertyValue()
	parentRule	Item()
		removeProperty()
		setProperty

DOM Interface: Acceso a estilos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los estilos insertados en el documento:

Objeto CSSStyleDeclaration:

```
<h2 id="h_uno" style="color:white;">Estilos "inline"</h2>
<ul id="ul_uno"></ul>

<h2 id="h_dos">Estilos "outline"</h2>
<ul id="ul_dos"></ul>
```

```
document.getElementById("h_uno").style.backgroundColor= "black";

const estilo_h_uno= document.getElementById("h_uno").style;
const estilo_h_dos= document.getElementById("h_dos").style;

txt_1= showStyles(estilo_h_uno);
document.getElementById("ul_uno").innerHTML= txt_1;

txt_2= showStyles(estilo_h_dos);
document.getElementById("ul_dos").innerHTML= txt_2;
```

```
<style>
  #h_dos {
    color: orange;
    background-color: black;
  }
</style>
```

```
function showStyles(cssStyle){

    let txt= "";
    for (let i=0; i<cssStyle.length; i++){
        estilo= cssStyle.item(i);
        valor= cssStyle.getPropertyValue(estilo);
        txt+= "<li>" + estilo + ": " + valor + "</li>";
    }

    return(txt);
}
```

Estilos "inline"

- color: white
- background-color: black

Estilos "outline"

DOM Interface: Acceso a estilos

HTML DOM define métodos y propiedades para acceder/modificar cada uno de los estilos insertados en el documento:

Objeto CSSStyleDeclaration:

```
<h2 id="h_uno" style="color:white;">Estilos
<ul id="ul_uno"></ul>

<h2 id="h_dos">Estilos "outline"</h2>
<ul id="ul_dos"></ul>
```

```
<style>
  #h_dos {
    color: orange;
    background-color: black;
  }
</style>
```

```
function showStyle(cssStyle, estilo){

    let txt= "";
    valor= estilo_h_dos.getPropertyValue(estilo);
    txt+= "<li>" + estilo + ": " + valor + "</li>";

    return(txt);
}
```

```
document.getElementById("h_uno").style.backgroundColor= "black";

const el_uno= document.getElementById("h_uno");
const el_dos= document.getElementById("h_dos");

const estilo_h_uno= window.getComputedStyle(el_uno);
const estilo_h_dos= window.getComputedStyle(el_dos);

txt_1= "<ul>";
txt_1+= showStyle(estilo_h_uno, "color");
txt_1+= showStyle(estilo_h_uno, "background-color");
txt_1+= "</ul>";
document.getElementById("ul_uno").innerHTML= txt_1;

txt_2= "<ul>";
txt_2+= showStyle(estilo_h_dos, "color");
txt_2+= showStyle(estilo_h_dos, "background-color");
txt_2+= "</ul>";
document.getElementById("ul_dos").innerHTML= txt_2;
```

Estilos "inline"

- color: rgb(255, 165, 0)
- background-color: rgb(0, 0, 0)

Estilos "outline"

- color: rgb(255, 165, 0)
- background-color: rgb(0, 0, 0)

DOM Interface: Acceso a eventos

Los eventos ocurren cuando los usuarios realizan algún tipo de acción, con el teclado o el ratón: pulsar ratón, moverse por un formulario, cambiar el tamaño de la ventana, cargar la página, etc

Cuando un evento ocurre, esa acción es utilizada para lanzar una función particular, que es programable.

El evento desencadenado se almacena en un objeto de la clase *Event.* Algunos de sus métodos y propiedades son:

Clase *Event*		
Propiedades	.type	Tipo de evento
	.target	Objeto que lo dispara
Métodos	.preventDefault()	Detiene acción por defecto
	.stopPropagation()	Detiene propagación en “burbuja”

DOM Interface: Acceso a eventos

Los eventos ocurren cuando los usuarios realizan algún tipo de acción, con el teclado o el ratón: pulsar ratón, moverse por un formulario, cambiar el tamaño de la ventana, cargar la página, etc

Cuando un evento ocurre, esa acción es utilizada para lanzar una función particular, que es programable.

Formas de ligar un evento a un elemento:

- Mediante atributos de eventos.
- Mediante manejadores de eventos DOM.
- Mediante "*listeners*" de eventos DOM level 2.

DOM Interface: Acceso a eventos

Formas de ligar un evento a un elemento:

- Mediante atributos de eventos.- habitual en *código antiguo*, pero no recomendada → No separa *javascript* del HTML

<element onEventName= “functionName()”>

```
<h1 id="cabecera"> USC </h1>
<p onclick="changeColor()">Pulsa para cambiar color a la cabecera</p>
```

```
function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}
```

USC

Pulsa para cambiar color a la cabecera

USC

Pulsa para cambiar color a la cabecera

DOM Interface: Acceso a eventos

Formas de ligar un evento a un elemento:

- Mediante manejadores de eventos DOM → Sólo permite asociar una función por evento.

element.onEventName= “functionName()”>

```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
```

```
document.getElementById("pulsar").onclick= changeColor;

function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}
```

USC

Pulsa para cambiar color a la cabecera

USC

Pulsa para cambiar color a la cabecera

DOM Interface: Acceso a eventos

Formas de ligar un evento a un elemento:

- Mediante "*listeners*" de eventos DOM level 2 → Forma actual de manejo de eventos. Permite asociar varias funciones por evento.

<element.addEventListener(eventName, functionName()>

```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
```

```
document.getElementById("pulsar").addEventListener("click", changeColor);
document.getElementById("pulsar").addEventListener("click", changeText);

function changeColor() {
  document.getElementById("cabecera").style.color= "red";
}

function changeText() {
  document.getElementById("cabecera").style.textDecoration= "underline";
}
```

USC

Pulsa para cambiar color a la cabecera

USC

Pulsa para cambiar color a la cabecera

DOM Interface: Acceso a eventos

Formas de ligar un evento a un elemento:

- Mediante "*listeners*" de eventos DOM level 2 → Forma actual de manejo de eventos. Permite asociar varias funciones por evento.

<element.addEventListener(eventName, functionName()>

```
<h1>Acceso a eventos</h1>
<h3>Pulsar para ver resultado:</h3>
<p id="uno"></p>
```

```
document.addEventListener("click", function() {
    miFunction(6, 7);
});

function miFunction(a, b) {
    let c= a * b;
    document.getElementById("uno").innerHTML= c;
}
```

Acceso a eventos

Pulsar para ver resultado:

Acceso a eventos

Pulsar para ver resultado:

42

DOM Interface: Acceso a eventos

Flujo de eventos.- cuando un evento ocurre en una etiqueta interior a otras, el evento se propaga hacia las etiquetas que la contienen (*false*) o viceversa (*true*).

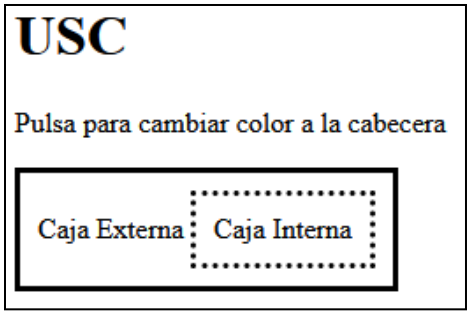
```
<h1 id="cabecera"> USC </h1>
<p id="pulsar">Pulsa para cambiar color a la cabecera</p>
<div id='out'>
   Caja Externa
       <div id='in'>
            Caja Interna
       </div>
</div>
```

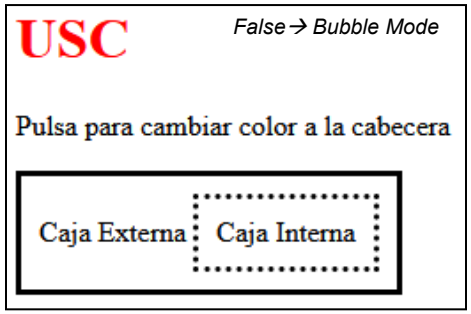
```
#out{
    border-style:solid;
    display:inline-block;
    padding:10px
}
#in{
    border-style:dotted;
    display:inline-block;
    padding:10px
}
```

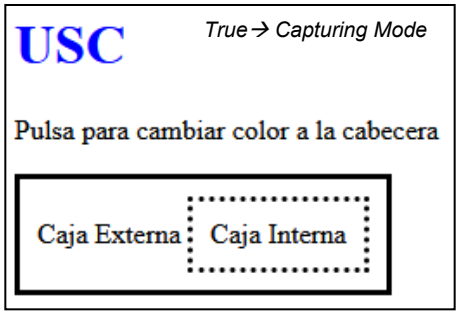
```
document.getElementById("out").addEventListener('click', changeRed, false);
document.getElementById("in").addEventListener('click', changeBlue, false);

function changeRed() {
  document.getElementById("cabecera").style.color= "red";
}

function changeBlue() {
  document.getElementById("cabecera").style.color= "blue";
}
```



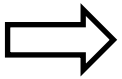


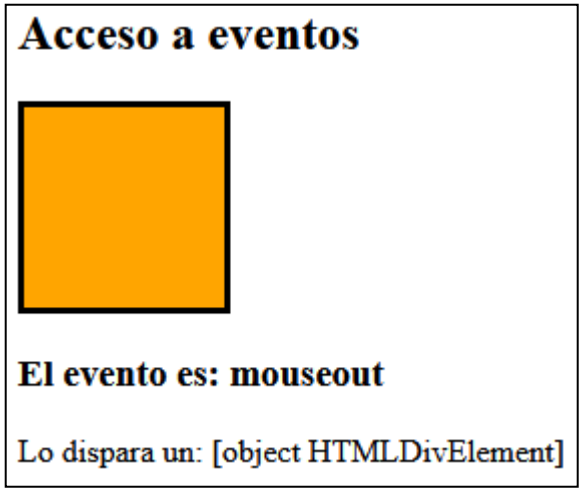


DOM Interface: Acceso a eventos

El acceso a los eventos se realiza a través del parámetro introducido en la función invocada por el “listener”.

```
<h2>Acceso a eventos</h2>
<div></div>
<h3></h3>
```






```
const box= document.getElementsByTagName("div")[0];

box.addEventListener("click", muestraEvento);
box.addEventListener("dblclick", muestraEvento);
box.addEventListener("mouseover", muestraEvento);
box.addEventListener("mouseout", muestraEvento);
```

```
function muestraEvento(e){
    txt_1= "El evento es: " + e.type;
    document.getElementsByTagName("h3")[0].textContent= txt_1;

    txt_2= "Lo dispara un: " + e.target;
    document.getElementsByTagName("p")[0].textContent= txt_2;
}
```

HTML Javascript APIs.

API (*Application Programming Interface*).- mecanismo que permite a un programa interaccionar con otros programas y/o scripts mediante el acceso a sus datos, para crear funcionalidades complejas de una manera simple.

JavaScript tiene muchas APIs construidas sobre su núcleo (no son parte del lenguaje), proporcionando nuevas funcionalidades adicionales.

Se dividen en dos categorías:

- **APIs de navegador.-** integradas en el navegador, permiten exponer datos del navegador y del entorno informático circundante y hacer cosas complejas con ellos → Acceso al documento (API DOM), Audio/Vídeo (Web Audio, WebRTC), Manipulación de gráficos (Canvas, WebGL), Geolocalización, Storage, …

- **APIs de terceros.-** no están incluídas por defecto en el navegador, y por ello es necesario obtener el código e información desde algún lugar de la Web → Facebook, Twiter, Google Maps, Youtube, …

HTML Javascript APIs.

API (*Application Programming Interface*).- mecanismo que permite a un programa interaccionar con otros programas y/o scripts mediante el acceso a sus datos, para crear funcionalidades complejas de una manera simple.

Para utilizar un programa externo, necesitamos saber:

- ¿Qué hace?
- ¿Cómo preguntarle qué hace?
- ¿Cuál es el formato de su respuesta?

HTML Javascript APIs.

API (*Application Programming Interface*).- mecanismo que permite a un programa interaccionar con otros programas y/o scripts mediante el acceso a sus datos, para crear funcionalidades complejas de una manera simple.

HTML5 APIs	Descripción
geolocation	Permite acceso a la localización geográfica del usuario
localStorage	Almacena información en el navegador, y la mantiene cuando la ventana del navegador se cierra
sessionStorage	Almacena información en el navegador, pero no la mantiene cuando la ventana del navegador se cierra
history	Permite acceso al historial de páginas visitadas por el navegador
console	Permite acceso a la consola de depuración (*debug*), del navegador

HTML Javascript APIs.

API geolocation

Permite averiguar la posición del usuario, en términos de coordenadas geográficas de latitud y longitud, siempre que éste proporcione el permiso para compartir esa información con la página Web.

Se implementa con el objeto *geolocation*, que es una propiedad del objeto *navigator*.

HTML Javascript APIs.

API geolocation

```
<body>
    <h2>Geolocalización</h2>
    <p id="geoLoc"></p>
</body>
```

Geolocalización

Posición actual: (42.8834816, -8.5426176)

```
<script>
function fnc(pos) {

  let x= "Posición actual: " +
  "(" + pos.coords.latitude +
  ", " + pos.coords.longitude + ")";

  document.getElementById("geoLoc").innerHTML= x;
}
navigator.geolocation.getCurrentPosition(fnc);
</script>
```

HTML Javascript APIs.

APIs localStorage y sessionStorage

- Permiten almacenamiento local de datos que pueden ser compartidos por las diferentes páginas del sitio web, en el lado cliente.
- Los datos son almacenados en el navegador.
- Es un mecanismo alternativo a las cookies.

Existen dos objetos diferentes:

- localStorage → los datos se almacenan sin fecha de expiración.
- sessionStorage → los datos se almacenan para la sesión → Se pierden al cerrar la ventana del navegador.

HTML Javascript APIs.

APIs localStorage y sessionStorage

Existen dos formas de almacenar los datos:

- Mediante el método setItem().

```
localStorage.setItem(nombreDato, valorDato);
```

- Usando el dato como atributo y dándole un valor

```
localStorage.nombreDato= valorDato;
```

HTML Javascript APIs.

APIs localStorage y sessionStorage

Existen dos formas de recuperar los datos:

- Mediante el método getItem().

```
valorDatos= localStorage.getItem(nombreDato);
```

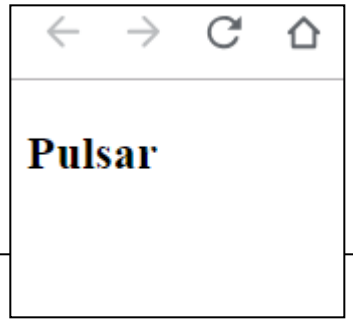
- Usando el dato como atributo y devolviéndolo valor

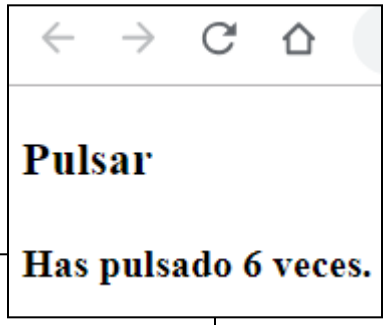
```
valorDato= localStorage.nombreDato;
```

HTML Javascript APIs.

APIs localStorage y sessionStorage

```
<body>
    <h3 onclick="nClicks()">Pulsar</h3>
    <h4 id="resultado"></h4>
</body>
```






```
<script>
function nClicks() {
    if(typeof(Storage) !== "undefined") {
        if (localStorage.n)
            localStorage.n= Number(localStorage.n) + 1;
        else
            localStorage.n= 1;
        document.getElementById("resultado").innerHTML = "Has pulsaado " + localStorage.n + " veces.";
    }
}
</script>
```

HTML Javascript APIs.

API console

Ofrece funcionalidad que permite a los desarrolladores realizar tareas de depuración, como registrar mensajes o los valores de las variables en puntos determinados del código, o cronometrar el tiempo que tarda en completarse una operación.

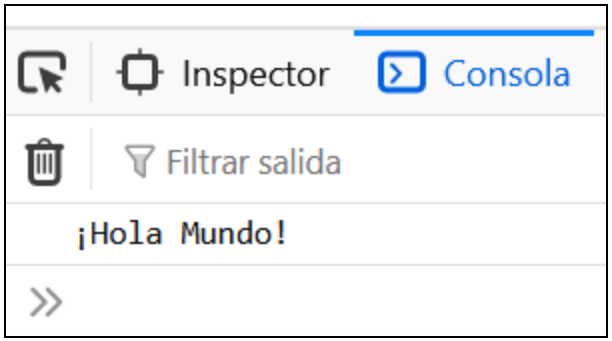
Se accede a través del objeto *console* que contiene muchos métodos con los que poder realizar tareas rudimentarias de depuración, generalmente centradas en el registro de varios valores en la *Consola Web* del navegador.

El método más utilizado es *console.log(),* que se utiliza para registrar el valor actual contenido dentro de una variable específica.

HTML Javascript APIs.

API console

```
<script>
  console.log("¡Hola Mundo!");
</script>
```



Desarrollo de Aplicaciones Web

JavaScript. Introducción.

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Pablo García Tahoces